AI IN ENGINEERING WORKFLOW

AI Usage Case Study

Reusable Hospital Department Architecture

Prakhar Upman

Full Stack MERN Developer

Project	Healthcare / Hospital Website
Primary Stack	Next.js · React · JavaScript · Tailwind CSS
AI Tool	ChatGPT
AI Role	Architecture exploration · Debugging · Development acceleration
Focus	Scalability · Reusability · Maintainability · UX

This case study documents how AI was integrated into a real software-development workflow to solve a maintainability and scalability problem while keeping engineering decisions, implementation, testing, and validation under developer control.

1. The Challenge / Problem

While developing a healthcare/hospital website, I needed to create pages for multiple medical departments such as Cardiology, Dermatology, Orthopedics, Neurology, and other services. The initial approach involved repeated page-level UI and logic, which created unnecessary duplication.

As the number of departments increased, this approach created several engineering concerns:

• Repeated React/Next.js markup and component logic across department pages.

• Higher maintenance effort when a common UI section needed to change.

• Greater risk of inconsistent layouts or behavior between departments.

• More development effort required to add a new department.

• Dynamic navigation and route handling needed a cleaner, scalable structure.

Engineering Goal

The goal was to separate **department-specific content** from **shared presentation logic**, allowing one reusable component to render many department pages through dynamic routing and structured data.

2. AI Tool Used

ChatGPT was used as an AI development assistant throughout the workflow for architecture exploration, debugging, code review-style feedback, implementation suggestions, and UI/UX refinement.

Use Case	How AI Assisted
Architecture	Explored reusable component and data-driven page structures.
Debugging	Analyzed specific Next.js routing, import, and rendering errors.
Implementation	Suggested implementation patterns and code changes.
UI/UX	Generated ideas for responsive layouts, cards, navigation, and visual hierarchy.
Validation	Helped identify edge cases and possible causes when behavior differed from expectations.

3. Prompting & Problem-Solving Approach

I used an iterative prompting process rather than asking AI to generate the entire application. I supplied the relevant requirements, project structure, code snippets, and exact error messages, then evaluated the suggestions before implementing them.

01 — Understand: Identify the actual engineering problem and define the desired architecture.

02 — Break Down: Separate routing, reusable UI, data, navigation, and department-specific content.

03 — Prompt: Give AI concrete context, existing code, requirements, and expected behavior.

04 — Evaluate: Review the suggested solution against the existing project and technical requirements.

05 — Implement: Apply the selected approach in the codebase.

06 — Test: Run the application and verify routes, UI, data, and responsive behavior.

07 — Refine: Return specific errors or edge cases to AI and iterate until the implementation works as intended.

Example Prompt

> “I have multiple department pages in my Next.js application with repeated UI and logic. Help me design a reusable architecture using dynamic routes, a shared DepartmentPage component, and centralized department data. Explain the folder structure and how the dynamic slug should map to the department data.”

Debugging Prompt Example

> “My Dermatology page shows ‘Department Not Found’ even though the department exists in my JSON data. Here is my dynamic route, JSON key, and navigation code. Identify the mismatch and explain exactly what needs to change.”

4. Architecture: Before → After

Before — Repeated Page Approach

```
app/
■■■ services/
■ ■■■ cardiology/page.jsx
■ ■■■ dermatology/page.jsx
■ ■■■ orthopedics/page.jsx
■ ■■■ neurology/page.jsx
■■■ components/
```

Each page could contain similar sections, styling, animations, statistics, cards, and layout logic. Updating a common section required changes in multiple files.

After — Data-Driven Reusable Approach

```
app/
■■■ services/
■ ■■■ [slug]/
■ ■ ■■■ page.jsx
■ ■■■ components/
■ ■ ■■■ DepartmentPage.jsx
■ ■■■ data/
■ ■■■ departments.json
■■■ components/
■■■ Navbar.jsx
```

The dynamic route receives the department slug, retrieves the matching data object, and passes it into the reusable DepartmentPage component.

Navbar	→	**Dynamic URL**	→	**Department Data**	→	**Reusable Page**
Department Link	→	**/services/cardiology**	→	**departments.json**	→	**DepartmentPage**

5. Implementation Pattern

A simplified representation of the final pattern is shown below. The key design principle is that content is data-driven and presentation is reusable.

```
// app/services/[slug]/page.jsx

import departments from "../data/departments.json";
import DepartmentPage from "../components/DepartmentPage";

export default async function Department({ params }) {
const { slug } = await params;
const department = departments[slug];

if (!department) {
return Department Not Found;
}

return ;
}
```

Why this pattern matters

• **Reusability:** one presentation component can serve many departments.

• **Maintainability:** common UI changes can be made in one place.

• **Scalability:** new departments can primarily be added as structured data.

• **Consistency:** departments share the same design and interaction patterns.

• **Separation of concerns:** content/data is separated from presentation logic.

6. Final Outcome / Impact

Area	Outcome
Code Structure	Moved toward reusable, data-driven department rendering instead of repeated page implementations.
Maintainability	Common presentation changes can be handled centrally.
Scalability	Adding departments becomes a data-and-route task rather than creating an entirely new page design.
Debugging	AI-assisted diagnosis shortened the feedback loop for routing and implementation issues.
Development Workflow	Less time spent on repetitive work and more focus on architecture, requirements, UX, and validation.

7. Developer Ownership & Validation

AI was used to accelerate exploration and implementation, but I remained responsible for understanding the proposed solution, integrating it into the codebase, testing the result, and deciding whether a suggestion was appropriate.

• I provided project-specific context rather than relying on generic generated code.

• I reviewed AI-generated suggestions before integrating them.

• I tested changes locally and investigated unexpected behavior.

• I used concrete error messages and code context for debugging.

• I retained responsibility for architecture, implementation choices, and final validation.

8. Key Takeaway

The most valuable use of AI in this project was not simply generating code. It was using AI to accelerate **problem decomposition, architecture exploration, debugging, and iteration**. The resulting workflow allowed me to move faster while maintaining engineering ownership and focusing on scalable, maintainable software.

Technology Context: Next.js · React · JavaScript · Tailwind CSS · Dynamic Routing · Reusable Components · JSON-driven Content